

Deep Implicit Layers for Smooth Implied Volatility Surface Interpolation*

✉ Rafael Zimmer

Institute of Mathematical Science and Computing
University of São Paulo - Brazil
rafael.zimmer@usp.br

Abstract—We implement a neural network architecture based on deep implicit layers with the objective of optimizing the parameters of multiple SABR (stochastic alpha, beta, rho) models of implied volatility surfaces in the context of financial options. The parameter values for the models are fitted on daily data points and used for generating smooth interpolated points in the surface. The motivation behind our approach is to reduce arbitrage opportunities when generating smooth interpolation and extrapolation of the surface. The proposed architecture determines the parameters of the SABR model that optimize for minimal statistical arbitrage opportunities in observed contracts. We fit the model using a dataset of historical listed option contracts as well as the calculated implied volatility for these contracts. The innate ability of transformer architectures to process observations of sequences with non-fixed lengths, i.e., for contracts with varying expiration dates is used to choose candidate points that best summarize the observed data. The surfaces generated with our network are then compared to a baseline linear interpolation model.

Index Terms—Neural Networks, Implied Volatility, Transformers, Quantitative Methods, Implicit Layers.

to all possible strike prices and maturities are not directly observable, as not all contracts are traded daily. Ensuring a way to obtain these values with confidence and using only readily available data is a crucial task for quantitative analysts [12]. Usual strategies for estimating these values are predominantly based on linear or spline interpolation algorithms [9] or stochastic differential equation models [10] that use numerical methods to approximate the values from existing listed contracts. Both methods try to approximate the correct value for the implied volatility for each pair of time to maturity and strike price. This approach allows visualizing the model results in the form of a surface, where the $\mathbf{XY}$ axes are the maturity and strike pairs and the $\mathbf{Z}$ axis is the IV . This surface, called the **implied volatility surface** (IVS), noted as S is therefore a function of the maturity time t and the strike price K ,

$$S := f(t, K), \quad \text{where } f: \mathbb{R}^2 \rightarrow \mathbb{R}. \quad (1)$$

I. INTRODUCTION

A. Options and Implied Volatility

Option contracts are one of the most important types of derivative contracts in the context of financial markets. A derivative is a contract traded with respect to another asset (also called the underlying asset) and an option contract allows the buyer of the contract to sell or buy the specified asset from the seller at a predefined future date. The pricing equations for these derivatives typically require knowing the time to maturity of the contract, the price of the underlying asset to be traded, the market's current risk-free rate, and the price at which the asset will be executed at maturity [1]. The underlying asset's volatility is also an important parameter to calculate the derived contract price, but knowledge of its true future value is not possible, so typically the market's estimate of the future volatility is used. This estimate, called the implied volatility (IV or σ_{iv}) is usually obtained by solving numerically for the volatility in the pricing equation using market quoted prices.

Having access to the implied volatility values allows investors to make sales at the prices consistent with the current expected contract price, that is, the no-arbitrage price according to the market. However, the IV values corresponding

Ensuring accurate predictions of the IVS is crucial, as even small errors can lead to incorrect pricing and consequently large losses of profit or even negative return [8] given the leveraging capabilities of options when compared to non-derivative securities. Classic models such as linear interpolation, polynomial fitting, and *splines* not only lack flexibility for extrapolation, but also generate either non-smooth surfaces, arbitrage opportunities, or both [8].

B. Contributions of the Presented Work

In this paper, we propose an approach to address the challenges of interpolation and extrapolation of the IVS, focusing on reducing arbitrage opportunities. The chosen method uses a stochastic differential equation solution approach, specifically the *Stochastic Alpha, Beta, Rho* (SABR) model. The parameters α, ρ, ν of the SABR model are obtained through the Broyden-Fletcher-Goldfarb-Shanno (BFGS) quasi-Newton non-linear optimization algorithm [2, 4, 5, 19]. This step is commonly performed using only a subset of the daily observed prices [13], due to the fact that some points may contain unreliable prices of illiquid contracts that do not match the real prices of the derivative, and their usage in the optimization step increases the variance of the final model. To solve this difficulty, the points used as input for the BFGS algorithm are often manually filtered according to criteria set by human

*This work was supported by the São Paulo State Research Support Foundation (FAPESP).

experts [6]. Rather than manually selecting these candidate points, the use of models capable of receiving point clouds and returning a score, such as transformers, to select the best points automatically has been proposed by Hagan et al. [6] instead. The main contribution of this research will be the adaptation of *neural implicit layers*, a physics-inspired variation of fully connected linear layers that outputs a solution to a fixed point equation, to be used together with the transformer architecture. This approach is an implicit fixed-point solver variation for the scoring model proposed by Kim et al. [13], aiming to enforce a zero arbitrage condition in the optimization process. This type of layer has been shown to perform well representing smooth surface densities and demonstrated state-of-the-art performance in function fitting tasks similar to the one presented here [11, 15], all while being computationally faster for solving interpolation tasks than linear layer models with similar performance [15]. Additionally, transformers have also been shown to perform better at handling unordered point clouds over time compared to recurrent neural layers [14] for IVS interpolation tasks, making them a suitable choice for the task at hand.

C. The Transformer Architecture

The aforementioned *SABR* model is a stochastic volatility model used to obtain smooth surfaces, i.e., continuous and infinitely differentiable at all points [7]. In the context of this work, a score will be assigned for each candidate point of the daily listed options contracts, called the *summarized suitability score* (*SS score*) by Kim et al. [13]. This score corresponds to the suitability of the candidate point to summarize the characteristics of the generated surface, i.e., how feasible it is to reduce the number of candidates when optimizing the values for one of the parameters of the *SABR* model without altering the resulting surface shape or its smoothness¹. Then, the highest scoring points — and therefore with the best representation of the surface — are selected and used as input for the BFGS algorithm. This process aims to remove outlier points that might introduce variance, and consequently arbitrage opportunities [13], in the optimization process for each of the *SABR* models².

D. Fixed-Point problems

Differently from the usual function approximation tasks, a fixed point problem is instead defined by conditions that the output must satisfy. For explicit functions, a given output z is computed directly from the input x using a function f , such that $z = f(x)$. In contrast, an implicit function g is such that it depends on both the input x and the expected output z , where z is a solution for the equation:

$$g(x, z) = 0 \quad (2)$$

¹The use of a per-point score is similar to what the principal components of a collection of points is when using the Principal Component Analysis (PCA) technique.

²A separate model is fitted for each unique strike price in a given day. Section II-A covers additional details on fitting the *SABR* model at each unique strike price.

Formulating the problem as a fixed point equation can be used for a multitude of problems, including finding fixed points in recurrent networks, solutions to differential equations leading to Neural ordinary differential equations, and several physics optimization problems [3]. The implicit formulation offers several practical advantages, primarily the separation of the solution method from the layer definition which can be useful in maintaining interpretability of the resulting model [11] and as a way to enforce constraints in the optimization process.

Moreover, the resulting implicit layer has significant benefits, specifically in deep learning and automatic differentiation frameworks [15, 18]. Automatic differentiation methods store the entire computation graph, which can be memory-intensive. With implicit layers, however, the implicit function theorem allows computing gradients directly at the solution point and therefore storing intermediate variables becomes redundant [18]. This improves memory efficiency, making implicit layers particularly advantageous when substituting existing layers within large deep learning models [16].

E. Anderson Acceleration for Implicit Layers

The Anderson Acceleration (AA) method is a technique used to accelerate the convergence of fixed-point iterations. It is particularly useful when the fixed-point iteration is slow to converge or when the fixed-point iteration is used in a deep learning context. Instead of using the current fixed-point iteration as the next guess as with the usual fixed-point iteration, a linear combination of the previous guesses is used to extrapolate the next, according to the following expression:

$$x_{k+1} = \beta_k x_k + (1 - \beta_k) x_{k-1}$$

where $\beta_k > 0$ is the mixing factor (also called relaxation parameter) used to determine the extrapolation weight. Two additional hyperparameters are used: the damping factor λ used to stabilize the fixed-point iteration; and a memory size m that determines the number of previous computed points to use in the extrapolation. We use a neural layer variation of the Anderson Acceleration method, which then becomes the main focus of this work: the implicit layer. Our proposed architecture uses the AA method to solve the fixed-point within in the implicit layer, according to the Algorithm 1 for our implementation of the AA method, and the layer weights are adjusted using backpropagation.

The final deep equilibrium model used within the implicit layer uses the Anderson Acceleration algorithm to solve a fixed-point equation, and the linear weights are used together with activation functions to compute the layer output, as shown in Algorithm 2. This output is then used as both the input for the next layer, or the final solution to the fixed-point equation if the layer is last in the network. When used to generate the IVS, the output score of the transformer is used as the input for the implicit layer, and a MAE error with regard to the observed data points is used as the objective function within the implicit layer to minimize the arbitrage opportunities in the generated surface.

Algorithm 1 Equilibrium Model with Anderson Acceleration

Require: Function f , initial point x_0 , memory size m , damping factor λ , maximum iterations K , tolerance ϵ , mixing factor β

```

1: Initialize  $X_0 = x_0, X_1 = F_0 = f(x_0), F_1 = f(F_0)$ 
2: for  $k = 2$  to  $K$  do
3:    $j \leftarrow \min(k, m)$ 
4:    $G = F_{0,\dots,j} - X_{0,\dots,j}$ 
5:    $H = GG^\top + \lambda I$ 
6:   Solve  $H^{-1} \cdot y = \alpha$ 
7:    $\alpha \leftarrow \alpha_{1,\dots,n+1}$ 
8:    $X_k \leftarrow \beta \alpha^\top F_0 + (1 - \beta) \alpha^\top X_0$ 
9:    $F_k \leftarrow f(X_k)$ 
10:  Compute residual:

```

$$\text{residual} = \frac{\|F_k - X_k\|}{\|F_k\| + 10^{-5}}$$

```

11:  if residual  $< \epsilon$  then
12:    break
13:  end if
14: end for
15: return  $X_k$ 

```

Algorithm 2 Implicit layer forward pass with Anderson acceleration (AA).

```

1: function IMPLICITLAYER( $x, m, \lambda, \epsilon, \beta$ )
2:   function  $f(x) \leftarrow W \cdot x + b$ 
3:    $x \leftarrow \text{AA}(f, x, m, \lambda, \epsilon, \beta)$ 
4:    $y \leftarrow x \cdot W^\top + b$ 
5:   return  $y$ 
6: end function

```

II. IMPLEMENTATION OF A PARAMETRIC SABR MODEL AND IMPLICIT NEURAL NETWORK ARCHITECTURES

A. Defining the SABR Model Equations

The SABR model is a stochastic differential model, defined by the following equations in which the dynamics of the asset price and the corresponding volatility levels are used to calculate an approximation to the implied volatility where no contract is traded.

- **Alpha** (α): This is the initial volatility level.
- **Beta** (β): The elasticity parameter between the volatility of the asset and its price.
- **Rho** (ρ): Statistical correlation between the asset price and its volatility, also called *spot-volatility correlation*.
- **Volvol** (ν): The volatility of the volatility process itself.

The implied volatility function of the SABR model is then given by:

$$\sigma_{BS}(f, K) = \alpha \frac{(fK)^{\frac{1-\beta}{2}}}{1 + \left(\frac{(1-\beta)^2 \rho^2}{24} + \frac{(1-\beta)^2 (2-3\rho^2)}{1920} \right) (fK)^{1-\beta}} \quad (3)$$

In the context of this paper, multiple SABR models are used daily, one for each unique strike price per day. Three continuous functions are used to map from continuous maturities to three model parameters: α , ρ , and ν . This is done to allow for a smooth transition between the different strike levels, as using a single SABR model by itself is known to not be able to replicate observed implied volatility surfaces [7, 13]. Given the functions for the parameters $\alpha_t = f_\alpha(t, \mathbf{P})$, $\rho_t = f_\rho(t, \mathbf{Q})$, and $\nu_t = f_\nu(t, \mathbf{R})$ (values for β are fixed and remain unchanged), and the encoding variables $\mathbf{P}$, $\mathbf{Q}$, and $\mathbf{R}$ respectively, such that $\mathbf{P} \in \mathbb{R}^5$, $\mathbf{Q} \in \mathbb{R}^4$, and $\mathbf{R} \in \mathbb{R}^4$, the daily parameters for the SABR model are obtained by the following equations for any continuous value of t :

$$f_\alpha(t, \mathbf{P}) = P_0 + \frac{p_3}{P_4} \left(\frac{1 - e^{-P_4 t}}{P_4 t} \right) + \frac{P_1}{P_2} e^{-P_2 t} \quad (4)$$

$$f_\rho(t, \mathbf{Q}) = Q_0 + Q_1 t + Q_2 e^{-Q_3 t} \quad (5)$$

$$f_\nu(t, \mathbf{R}) = R_0 + R_1 t^{R_2} e^{R_3 t} \quad (6)$$

Therefore, the task becomes obtaining the encoding variables $\mathbf{P}$, $\mathbf{Q}$, and $\mathbf{R}$ for daily SABR models. Initially, we use the Broyden-Fletcher-Goldfarb-Shanno (BFGS) algorithm to fit the encoding variables that best minimize the mean absolute error (MAE) between the observed implied volatilities and corresponding SABR generated implied volatilities. The resulting SS scores are then calculated as the MAE between the fitted $f_\alpha(t, \mathbf{P})$, $f_\rho(t, \mathbf{Q})$, $f_\nu(t, \mathbf{R})$ and the fitted values for $\tilde{f}_\alpha(t, \tilde{\mathbf{P}})$, $\tilde{f}_\rho(t, \tilde{\mathbf{Q}})$, and $\tilde{f}_\nu(t, \tilde{\mathbf{R}})$, where $\tilde{\mathbf{P}}$, $\tilde{\mathbf{Q}}$, and $\tilde{\mathbf{R}}$ are the encoding variables obtained using a subset of the daily candidate according to a Monte Carlo selection detailed in Section II-B3, similarly to the selection a human trader would make when selecting the most relevant data points to fit the SABR parameters to reduce arbitrage opportunities [13]. A visual representation of a surface with an arbitrage opportunity is shown in Figure 3. Finally, the neural network is trained to predict the SS scores for each candidate point $\mathcal{S}_i^t \in \mathcal{S}^t$ from the daily observed input. The task can then be classified as a supervised regression task and a Root Mean Squared Error (RMSE) between the transformer predicted SS scores and those calculated using the Monte Carlo estimation is used as loss score³.

B. Proposed Architecture

As mentioned, transformers are capable of capturing complex dependencies in sequential non-fixed length data. Within our proposed architecture, we use a transformer encoder to approximate the SS scores for each candidate point $\mathcal{S}_i^t \in \mathcal{S}^t$ from the daily observed input.

³The RMSE between the SS scores is not to be confused with the MAE between the calculated SABR parameters. The first is used as the loss function when training the neural network model, while the second is used as the BFGS objective function when optimizing for the encoding variables that best replicate the observed daily surface.

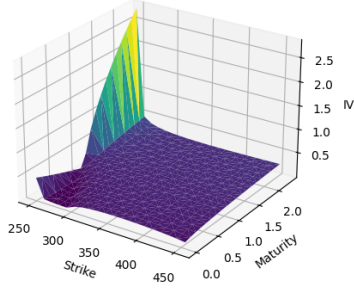


Fig. 1: Surface generated using SABR models fit on entire candidate set, resulting in an outlier peak.

1) *Details on the Chosen Transformer Encoder:* Unlike recurrent neural networks (RNNs) and convolutional neural networks (CNNs), transformers do not rely on a fixed-length data sequence. Instead, they use an attention mechanism, which allows each input element (or token) to consider the relationship with all other elements in the same sequence simultaneously. This is achieved through a block of layers using attention heads.

The attention mechanism accepts continuous data and calculates a value for different heads through the value (V), key (K), and query (Q) matrices, resulting in an output weighted by the importance of each element in the sequence [20, 22]. Specifically, for our problem we use the mechanism of self-attention, and therefore the three matrices are equal.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (7)$$

Transformers are classified as machine learning algorithms due to their ability to learn patterns and relationships from data, and were originally proposed for natural language processing tasks. In the context of our specific architecture, we use only the encoder block and adapt it to perform a regression on the SS scores. The hidden values are passed to fully-connected (FC) layers, and final implicit fixed-point iteration layers are used to obtain the scores as SS z-normalized values. The training epochs adjust the layer weights with an error function that minimizes the difference between its predictions and expected z-score values.

2) *Details on the Implicit Layer:* Using a Deep Equilibrium Model variation for the Implicit Layer, we adapt Equation 2 by using the following fixed point equation instead, where g is the MAE between the observed surface and the generated surface using the candidate set filtered by SS scores:

$$\begin{aligned} g\mathbf{W}(x, z^*) &= 0 \\ z^* &= f(z^*, x) \end{aligned} \quad (8)$$

in which f is an activation function, $\mathbf{W}$ are the layer weights and biases, and z^* is the target output of a batched non-normalized Jacobian to be used as the gradient function in our

code, given by the equation below. Finally, y are the hidden values as well as the final output scores.

$$\left(\frac{\partial z^*(\cdot)}{\partial(\cdot)}\right)^T y = \left(\frac{\partial f(z^*, x)}{\partial(\cdot)}\right)^T g\mathbf{W}$$

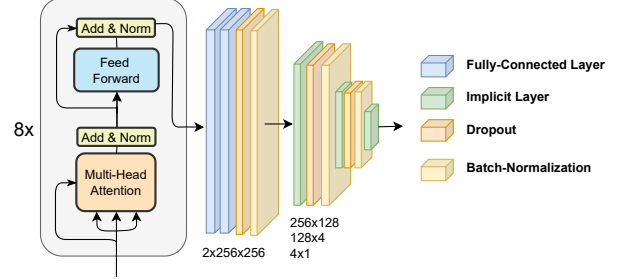


Fig. 2: Visualization of the Transformer Encoder and Implicit Layers used in the proposed architecture.

3) *Monte Carlo Estimation of Z-Scores:* As mentioned in Section II-A, the Monte Carlo estimation is used to estimate the synthetic target z-scores for the model. Instead of using the entire daily candidate set for calculating the encoded variables $\mathbf{P}$, $\mathbf{Q}$, and $\mathbf{R}$ for the SABR model, only a small subset of the candidate set is used as input to the *BFGS* algorithm to reduce surface volatility, resulting in $\tilde{\mathbf{P}}$, $\tilde{\mathbf{Q}}$, and $\tilde{\mathbf{R}}$. The SS score is then calculated as the mean absolute error (MAE) between the functions f_α , f_ρ , and f_ν using both encoded variable types as shown in Algorithm 3 and its normalized values are used as target values for our model. The choice of using a neural network approach for calculating the SS scores is motivated by the much higher computational cost of the Monte Carlo estimation, as well as the possibility of using previous observations to calculate the current day's scores.

This estimation using a finite number of subsets is necessary due to computational constraints, both in terms of time and memory. Finally, model training is performed until convergence to low RMSE values between predicted scores and estimated scores. After training, the model prediction substitutes the Monte Carlo estimation for the SS scores, and the best candidate set is selected and passed to the *BFGS* algorithm to obtain the final encoding variables for the SABR models when generating the daily surface.

III. METHODOLOGY AND RESULTS

A. Data Description and Pre-Processing

The dataset used for training was obtained from historical traded option contracts on the *SPY* asset from January 2021 to December 2022, with a total of 68 unique days. The dataset contains various columns on traded options, indexed by date, including strike prices, future contract prices, maturities, implied volatilities, among others. Relevant to our study, the principal columns used were:

- **Underlying:** Price of the underlying asset.
- **Strike:** Option's strike price.

Algorithm 3 Monte Carlo Estimation of Target Z-Scores

Require: Daily candidate set $\mathcal{S}^t$, percentage $p = 0.8$, number of subsets $k = 1e2$

```

1:  $\alpha^*, \rho^*, \nu^* \leftarrow BFGS(\mathcal{S}^t)$ 
2:  $SS_\alpha, SS_\rho, SS_\nu \leftarrow \mathbf{0}$ 
3: for candidate  $c$  in  $\mathcal{S}^t$  do
4:    $\mathcal{S}_i^t \leftarrow \text{Sample}(\mathcal{S}^t, k, p)$ 
5:    $\tilde{\mathbf{P}}, \tilde{\mathbf{Q}}, \tilde{\mathbf{R}} \leftarrow BFGS(\mathcal{S}_i^t)$ 
6:    $SS_{\alpha,c} \leftarrow \text{MAE}(f_\alpha(t, \tilde{\mathbf{P}}) - \alpha^*)$ 
7:    $SS_{\rho,c} \leftarrow \text{MAE}(f_\rho(t, \tilde{\mathbf{Q}}) - \rho^*)$ 
8:    $SS_{\nu,c} \leftarrow \text{MAE}(f_\nu(t, \tilde{\mathbf{R}}) - \nu^*)$ 
9: end for
10: z-normalize  $SS_\alpha, SS_\rho, SS_\nu$ 

```

- **Expiration:** Days remaining until the option’s expiration.
- **Implied Volatility:** Value for the implied volatility.
- **Date:** Traded date for the contract, used as the preprocessed dataset index.

Additionally, two columns were added to be used for the Monte Carlo estimation:

- **Maturity:** Calculated by dividing *Expiration* by 252 (working days in a year) to normalize the maturity time.
- **Risk-free rate:** Risk-free rate obtained from the Federal Reserve Economic Data (FRED) for the corresponding period.
- **Dividend:** Dividend yield paid quarterly by the SPY index.

B. Data Preprocessing

The data preprocessing involved several essential steps, predominantly to allow the use of the surface data directly in the transformer architecture and subsequent generation of the surface for historical data. Each of the trained transformers (one for each parameter α, ρ , or ν) takes four dimensions as input, which are:

- 1) Time to maturity;
- 2) The daily parameters α, ρ , or ν calculated using the *BFGS* optimization algorithm;
- 3) 1 if the parameter was calculated for the current day, 0 if it was calculated for a fixed set of maturities (top 7 most frequent maturities) using the previous day fitted parameters;
- 4) Parameter value calculated using the previous day’s $\mathbf{P}$, $\mathbf{Q}$ and $\mathbf{R}$ values for same strike and maturity.

The target scores were generated beforehand using the aforementioned Monte Carlo method with $p = 80\%$ and $k = 1 \times 10^2$. For training and testing we also observed that the choice of $\beta = 0.5$ performed best at minimizing the surface’s MAE.

C. Results and Surface Comparison

A baseline linear interpolation for the daily observed points was also used to compare our trained model’s surfaces by interpolating the points within a fixed grid of strikes and

maturity. No extrapolation was performed due to the nature of the linear model.

Our transformer model was trained on the dataset, with about 44k observations total resulting in 68 days. With an early stopping sensitivity of $\epsilon = 10^{-3}$, the training was run with a maximum epoch range of 10^3 using the Adam optimizer. For the implicit layer, we used the Anderson Acceleration algorithm with a maximum of 10 iterations, convergence threshold of $= 10^{-6}$, damping factor of $\lambda = 0.1$, and memory size of $m = 5$. The training time for the transformer model was approximately 8 minutes and 11 seconds, while the Monte Carlo preprocessing time was around 21 minutes and 5 seconds for the entire dataset. A batch size of 8 was used to avoid memory issues when training the implicit layer. The network architecture is a transformer encoder block with 8 attention heads, 2 linear layers with 256 hidden units and 2 implicit layers with 256 and 128 hidden units, respectively. An illustration of the architecture is shown in Figure 1, and a dropout rate of 0.1 was used for the dropout layers.

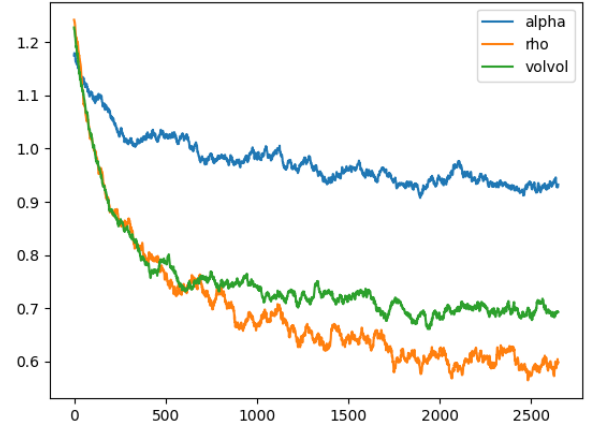


Fig. 3: Historical loss curve on training session data. Curves were cut for better visualization, with the largest episode being 2950 epochs for the SST_α (alpha) network.

Model	Test Loss (RMSE)	Train time (Transformer)	Preprocess Time (Monte Carlo)
SST_α	1.21	~4m21s	~7m50s
SST_ρ	0.72	~2m8s	~6m34s
SST_ν	0.75	~1m43s	~6m41s

TABLE I: Model training metrics for each of the three SST models.

For benchmarking, the execution time was measured using the Python `perf_counter` method and a heatmap of the model was executed on the entire dataset. Separate tests using a linear interpolation with the original points and with the implicit layer were performed to compare visually the generated surface’s error. The training and tests were performed using leave-one-out cross-validation (LOO), due to the small

amount of days in the dataset (68 days). The train and test losses shown in Table I are calculated using LOO, while the execution time shown below in Table II is calculated using the entire dataset. The RMSE values for the test data show that the model was able to generalize well to the test data, with the lowest RMSE value for the ρ parameter. The error values were calculated by comparing the generated surface with the original points, as mentioned in the previous section, and a visual comparison of the surfaces is shown in Figure 4.

Metric	Mean time	Standard Deviation
Execution time on entire dataset	229.1ms	41.2ms
Execution time per point	2ms	1ms

TABLE II: Model performance

IV. CONCLUSION

A. Summary

This work presents an architecture taking as inspiration approaches from both Kim et al. [13] and Kolter et al. [15] for the task of choosing a candidate set for fitting the parameters of daily SABR models using a neural network based on the transformer architecture (specifically the encoder element) with implicit deep equilibrium layers as the output components. The combination of the SABR model with the transformer architecture for candidate selection was used to replicate the patterns of the daily implied volatility surface, generating a smooth surface that reduces arbitrage opportunities through the solution of a fixed-point problem using the Anderson Acceleration algorithm within the implicit layers. The candidate scoring and selection process is performed automatically, instead of heuristically by human traders, the model allows for the fine-tuning using new daily observations to be taken into account when adjusting the SABR parameters for the next trading day.

B. Implications and Future Work

The main focus of this work was to merge the approach using the transformer and the implicit layers to replace the manual selection of scores for candidates, as well as show the possibility of future research focusing on High-Frequency Trading (HFT) arbitrage strategies and the reduction of execution speeds shown in Table II. Adapting the model to accept exogenous market factors besides the observed data, such as macroeconomic indicators, market indexes, and other financial instruments, could allow the model to better adapt to changing market regimes and is also a promising extension in future research.

REFERENCES

- [1] Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.
- [2] Charles G Broyden. The convergence of a class of double-rank minimization algorithms 1. general considerations. *IMA Journal of Applied Mathematics*, 6(1):76–90, 1970.
- [3] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. 2019. URL <https://arxiv.org/abs/1806.07366>.
- [4] Roger Fletcher. A new approach to variable metric algorithms. *Computer Journal*, 13(3):317–322, 1970.
- [5] Donald Goldfarb. A family of variable-metric methods derived by variational means. *Mathematics of Computation*, 24(109):23–26, 1970.
- [6] Patrick Hagan, Deep Kumar, Andrew Lesniewski, and Diana Woodward. Managing smile risk. *Wilmott Magazine*, 1:84–108, 01 2002.
- [7] Patrick Hagan, Andrew Lesniewski, and Diana Woodward. *Probability Distribution in the SABR Model of Stochastic Volatility*, volume 110, pages 1–35. 06 2015. ISBN 978-3-319-11604-4. doi: 10.1007/978-3-319-11605-1_1.
- [8] Patrick S. Hagan, Deep Kumar, Andrew Lesniewski, and Diana Woodward. Arbitrage-free sabr. *Wilmott*, 2014(69):60–75, 2014. doi: <https://doi.org/10.1002/wilm.10290>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/wilm.10290>.
- [9] Cristian Homescu. Implied volatility surface: Construction methodologies and characteristics, 2011.
- [10] Richard Jordan and Charles Tier. Asymptotic approximations to cev and sabr models. *SSRN Electronic Journal*, pages 1–38, May 2011. Available at SSRN: <https://ssrn.com/abstract=1850709> or <http://dx.doi.org/10.2139/ssrn.1850709>.
- [11] Kenji Kawaguchi. On the theory of implicit deep learning: Global convergence with implicit layers. 2021. URL <https://arxiv.org/abs/2102.07346>.
- [12] Bo-Hyun Kim, Daewon Lee, and Jaewook Lee. Local volatility function approximation using reconstructed radial basis function networks. In J Wang, Z Yi, JM Zurada, BL Lu, and H Yin, editors, *ADVANCES IN NEURAL NETWORKS - ISSN 2006, PT 3, PROCEEDINGS*, volume 3973 of *Lecture Notes in Computer Science*, pages 524–530, HEIDELBERGER PLATZ 3, D-14197 BERLIN, GERMANY, 2006. Univ Elect Sci & Technol China; Chinese Univ Hong Kong; Asia Pacific Neural Network Assembly; European Neural Network Soc; IEEE Circuits & Syst Soc; IEEE Computat Intelligence Soc; Int Neural Network Soc; Natl Nat Sci Fdn China; KC Wong Educ Fdn, Hong Kong, SPRINGER-VERLAG BERLIN. ISBN 3-540-34482-9. 3rd International Symposium on Neural Networks (ISSN 2006), Chengdu, PEOPLES R CHINA, MAY 28-31, 2006.

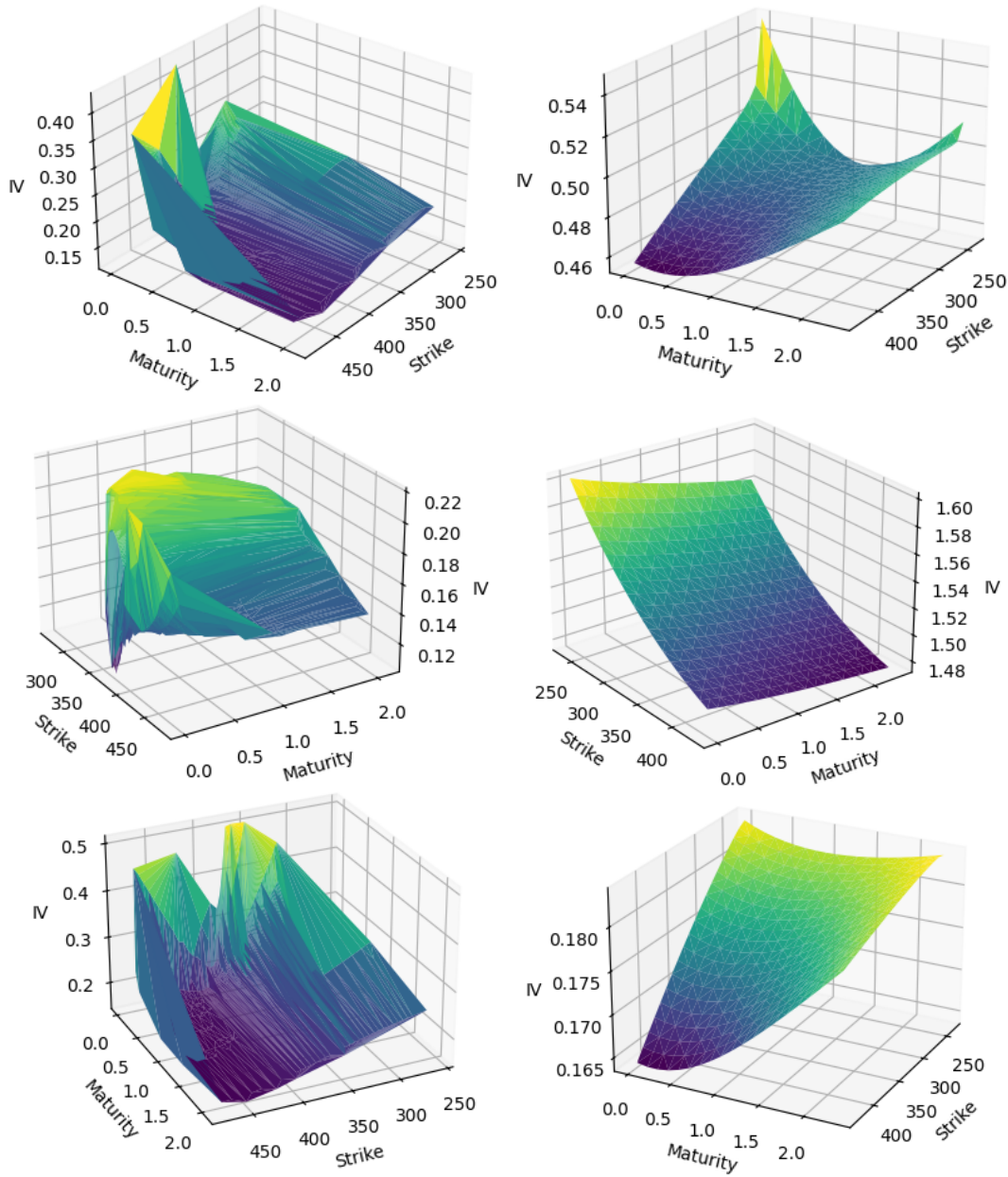


Fig. 4: Surfaces interpolated on test data using a linear interpolation model (left) compared to the SABR models fitted with the candidate set selected using the trained implicit transformer architecture (right).

- [13] Hyeonuk Kim, Kyunghyun Park, Junkee Jeon, Changhoon Song, Jungwoo Bae, Yongsik Kim, and Myungjoo Kang. Candidate point selection using a self-attention mechanism for generating a smooth volatility surface under the sabr model. *EXPERT SYSTEMS WITH APPLICATIONS*, 173, JUL 1 2021. ISSN 0957-4174. doi: 10.1016/j.eswa.2021.114640.
- [14] Soohan Kim, Seok-Bae Yun, Hyeong-Ohk Bae, Muhyun Lee, and Youngjoon Hong. Physics-informed convolutional transformer for predicting volatility surface. *QUANTITATIVE FINANCE*, 24(2):203–220, JAN 9 2024. ISSN 1469-7688. doi: 10.1080/14697688.2023.2294799.
- [15] Zico Kolter, David Duvenaud, and Matt Johnson. Deep implicit layers - neural odes, deep equilibrium models, and beyond, 2020. NeurIPS 2020 tutorial. Retrieved from <https://implicit-layers-tutorial.org/>.
- [16] Xuechen Li, Ting-Kam Leonard Wong, Ricky T. Q. Chen, and David Duvenaud. Scalable gradients for stochastic differential equations. 2020. URL <https://arxiv.org/abs/2001.01328>.
- [17] M. Ludkovski. Statistical machine learning for quantitative finance. *ANNUAL REVIEW OF STATISTICS AND ITS APPLICATION*, 10:271–295, 2023. ISSN 2326-8298. doi: 10.1146/annurev-statistics-032921-042409.

- [18] Stefano Massaroli, Michael Poli, Jinkyoo Park, Atsushi Yamashita, and Hajime Asama. Dissecting neural odes. 2021. URL <https://arxiv.org/abs/2002.08071>.
- [19] David F Shanno. Conditioning of quasi-newton methods for function minimization. *Mathematics of Computation*, 24(111):647–656, 1970.
- [20] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. 2023. URL <https://arxiv.org/abs/1706.03762>.
- [21] Homer F. Walker and Peng Ni. Anderson acceleration for fixed-point iterations. *SIAM Journal on Numerical Analysis*, 49(4):1715–1735, 2011. doi: 10.1137/10078356X.
- [22] Qingsong Wen, Tian Zhou, Chaoli Zhang, Weiqi Chen, Ziqing Ma, Junchi Yan, and Liang Sun. Transformers in time series: A survey. 2023. URL <https://arxiv.org/abs/2202.07125>.